Oblak Cloud Platform

Technical Documentation for the Dockerized Python Function Upload and Verification Platform

Danila Travkov
Vladimir Morozkin
Roman Avanesov

Field	Value
Application name	Oblak Cloud Platform
Primary API	FastAPI Cloud Server
CLI	Oblak CDK CLI
Verification	Bandit, Google Gemini LLM analysis, ClamAV
Container orchestration	Docker Compose
Runtime status	Function upload and verification are implemented. /run/{function_id} records an invoke request and returns HTTP 501. Production execution belongs to the Firecracker runner stage.

1. Application Overview

Oblak accepts Python functions, verifies them, stores them, and returns an invoke URL. The Cloud Server owns authentication, upload, persistence, verification, audit logging, listing, and deletion. The Firecracker runner is the separate execution-stage package.

- Bearer-token authentication through the CDK CLI.
- Multipart deployment of one .py file and optional requirements.txt.
- SQLite storage for source code, requirements, hashes, status, verification results, and invoke URLs.
- Fail-closed verification with Bandit, Gemini LLM review, and ClamAV.
- SQLite and JSONL audit events with hash-chain integrity.
- Docker Compose environment for local operation and tests.

2. High-Level Architecture

Component	Responsibility	Docker service
cloud-server	Authentication, upload, verification orchestration, SQLite persistence, audit logging, listing, deletion, and invoke URL placeholder.	cloud-server
cdk-cli	Login, deploy, list, and delete operations.	cdk
code-verifier	Bandit static analysis, Gemini LLM analysis, and ClamAV scanning.	code-verifier
firecracker-runner	Bundle validation, Firecracker payload preparation, audit logging, and dry-run validation.	firecracker-runner

3. Service Details

3.1 Cloud Server

Endpoint	Method	Purpose
/health	GET	API health and tool availability.
/auth/me	GET	Bearer token validation.
/auth/login	POST	Alternative token validation.
/functions	POST	Function upload, verification, storage, and invoke URL creation.
/functions	GET	Function listing for the authenticated user.
/functions/{function_id}	DELETE	Soft deletion.
/run/{function_id}	POST	Audited invoke placeholder returning HTTP 501.
/admin/antivirus/refresh	POST	Manual freshclam run.

3.2 CDK CLI

```
"dev-token" | docker compose run --rm -T cdk login
docker compose run --rm -T cdk deploy /workspace/examples/benign/handler.py
docker compose run --rm -T cdk list
docker compose run --rm -T cdk delete <function_id>
```

3.3 Code Verifier

Check	Tool	Blocking behavior
Static analysis	Bandit	Blocks medium and high severity findings.
LLM security review	Google Gemini	Flags destructive file operations, exfiltration, fork bombs, sandbox escapes, and suspicious network behavior.
Antivirus	ClamAV	Blocks known malware signatures and scan errors.

3.4 Firecracker Runner

The runner validates bundles, creates staging payloads, writes audit events, and supports dry-runs without KVM. Real execution requires Linux, /dev/kvm, Firecracker, a kernel image, a rootfs image, and host isolation controls.

4. Persistence and Audit

Data	Storage
SQLite database	cloud-data volume, /data/oblak_cloud.sqlite3.
Cloud audit JSONL	cloud-data volume, /data/oblak-cloud-audit.log.
ClamAV database	clamav-db volume, /var/lib/clamav.
CDK token config	cdk-home volume, /root/.cdk/config.json.
Firecracker dry-run output	firecracker-work volume, /workspace/.oblak.

- Function records store source, requirements, SHA-256 hashes, status, verification result, invoke URL, timestamps, and deletion marker.

- Audit records include actor, function id, request id, IP, outcome, details, previous hash, and event hash.

5. Security Requirements and Threat Controls

- Bearer token required for protected API operations.

- SHA-256 token hashes in SQLite; raw API tokens are not stored.

- Single .py upload plus optional requirements.txt.

- Path-component rejection for uploaded filenames.

- Upload size limits before temporary file scanning.

- VERIFIED status only after successful verification.

- Fail-closed behavior for Bandit, Gemini, and ClamAV failures.

- Production execution boundary: Firecracker microVM with jailer, cgroups, seccomp, no host mounts, read-only bundle drive, non-root guest user, and resource limits.

6. Configuration

```
cd C:\RazvojBezbednogSoftvera\RBS\project
Copy-Item cloud-server\env.example cloud-server\.env
notepad cloud-server\.env
```

Variable	Docker value	Purpose
GEMINI_API_KEY	cloud-server/.env	Gemini LLM verification.
OBLAK_DATABASE_PATH	/data/oblak_cloud.sqlite3	SQLite database path.
OBLAK_AUDIT_LOG_PATH	/data/oblak-cloud-audit.log	Cloud audit JSONL path.
OBLAK_PUBLIC_BASE_URL	http://localhost:8000	Returned invoke URL base.
OBLAK_BOOTSTRAP_USERNAME	alice	Bootstrap user.
OBLAK_BOOTSTRAP_TOKEN	dev-token	Development API token.
OBLAK_RUN_FRESHCLAM_ON_STARTUP	true	Startup ClamAV definition refresh.

7. Docker Services

Service	Image source	Runtime mode	Mounts
cloud-server	cloud-server/Dockerfile	Long-running API on port 8000	cloud-data, clamav-db
cdk	cdk-cli/Dockerfile	One-shot CLI	cdk-home, examples read-only
code-verifier	code-verifier/Dockerfile	One-shot tests	clamav-db, samples read-only
firecracker-runner	firecracker-runner/Dockerfile	One-shot dry-run	firecracker-work, examples read-only

8. End-to-End Test

```
cd C:\RazvojBezbednogSoftvera\RBS\project
.\scripts\e2e.ps1

# Reuse existing Docker volumes
.\scripts\e2e.ps1 -KeepState
```

- Builds all Docker images.
- Starts cloud-server and checks /health.
- Logs in with dev-token through the CDK CLI.
- Deploys the benign example function.
- Lists stored functions.
- Rejects the malicious example function.
- Checks /run/{function_id} returns HTTP 501.
- Deletes the benign function.

9. Manual Startup and CLI Examples

```
docker compose --profile tools build
docker compose up -d cloud-server
docker compose ps
Invoke-RestMethod http://localhost:8000/health
"dev-token" | docker compose run --rm -T cdk login
docker compose run --rm -T cdk deploy /workspace/examples/benign/handler.py
docker compose run --rm -T cdk list
docker compose run --rm -T cdk deploy /workspace/examples/malicious/handler.py
docker compose down
```

10. Component Test Commands

```
# Cloud Server API tests
docker compose run --rm --entrypoint python cloud-server -m pytest

# Code verifier tests
docker compose run --rm code-verifier

# Firecracker runner dry-run
docker compose run --rm firecracker-runner `
  --bundle /workspace/examples/hello `
  --event-file /workspace/examples/hello/event.json `
  --function-id hello `
  --dry-run
```

11. API Examples

```
curl.exe -H "Authorization: Bearer dev-token" http://localhost:8000/auth/me
curl.exe -H "Authorization: Bearer dev-token" http://localhost:8000/functions

curl.exe -X POST http://localhost:8000/functions ^
  -H "Authorization: Bearer dev-token" ^
  -F "file=@cloud-server/examples/benign/handler.py;type=text/x-python" ^
  -F "requirements=@cloud-server/examples/benign/requirements.txt;type=text/plain"

curl.exe -X POST -H "Authorization: Bearer dev-token" http://localhost:8000/run/<function_id>
```